

BCAC 0026: ANALYSIS OF ALGORITHM AND DATA STRUCTURES

Faculty Name

Dr. Vinod Jain, Assistant Professor, CEA

Mob : 9813078438, Email – vinod.jain@gla.ac.in

Seating : 329-B, AB-X, Third Floor

Stacks and Queues:

- **Stacks and Queues:** Implementing stack using array and linked list, Prefix, Infix and Postfix expressions, Utility and conversion of these expressions from one to another; Array and Linked representation of Queue, De-queue, Priority Queues.
- Mid term do not include Linked representation of Stack and Queue.

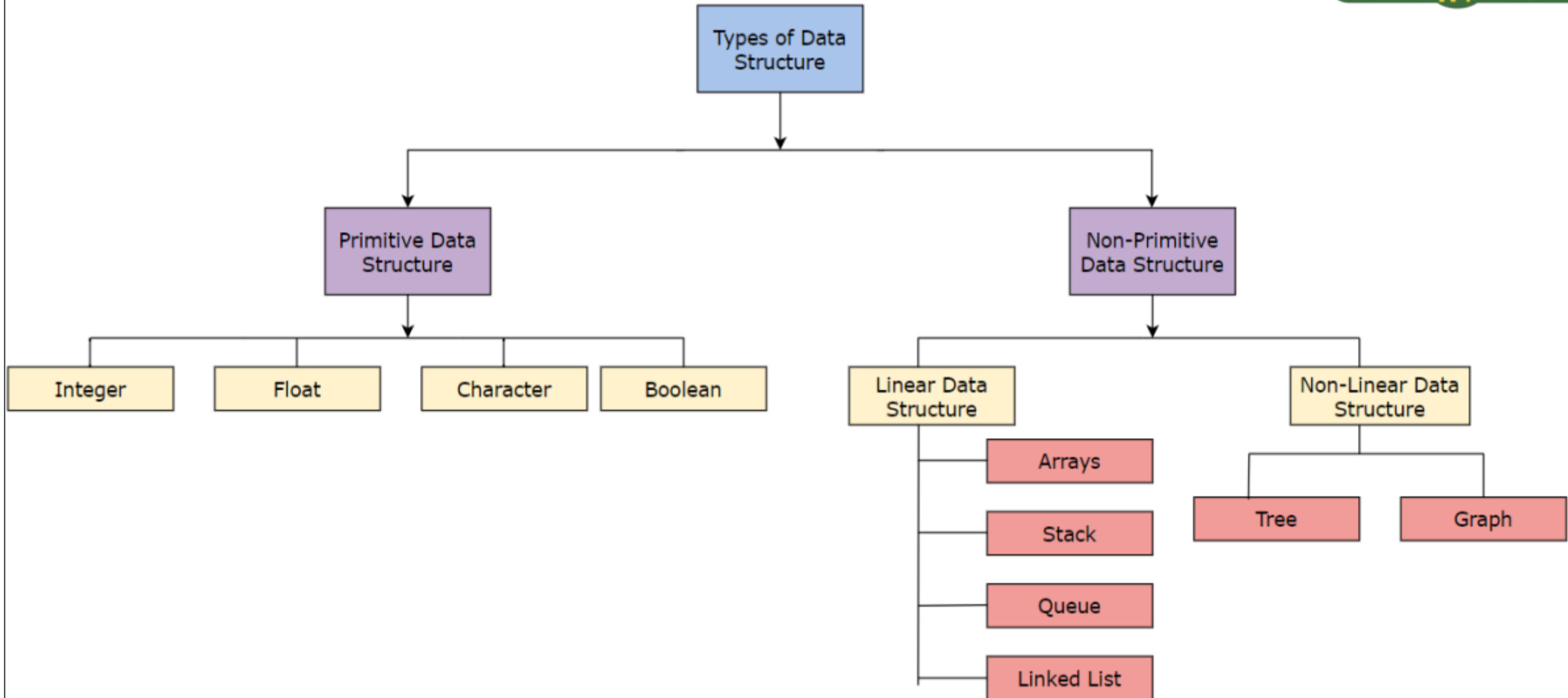
Stacks and Queues:

- Implementing stack using array **and linked list**
- Prefix, Infix and Postfix expressions
- Utility and conversion of these expressions from one to another
- Array and **Linked representation** of Queue
- De-queue, Priority Queues

Stacks and Queues:

- Implementing stack using array
- Prefix, Infix and Postfix expressions
- Utility and conversion of these expressions from one to another
- Array of Queue
- De-queue, Priority Queues

- Types of data structure:



Types of Data Structures

- Primitive data structure
- Non Primitive Data Structures

Primitive data structure

- Primitive data structure is a data structure that can hold a single value in a specific location.
- The examples of primitive data structure are float, character, integer and pointer.

The non-primitive data structure

- The non-primitive data structure is a kind of data structure that can hold multiple values either in a contiguous or random location.
- The non-primitive data types are defined by the programmer.
- The non-primitive data structure is further classified into two categories, i.e., linear and non-linear data structure.

A linear data structure

- A linear data structure has data elements connected to each other so that elements are arranged in a sequential manner and each element is connected to the element in front of it and behind it.
- This way, the structure can be traversed in a single run
- For Example
 1. Array
 2. Stack
 3. Queue
 4. Linked list

Non-linear data structure

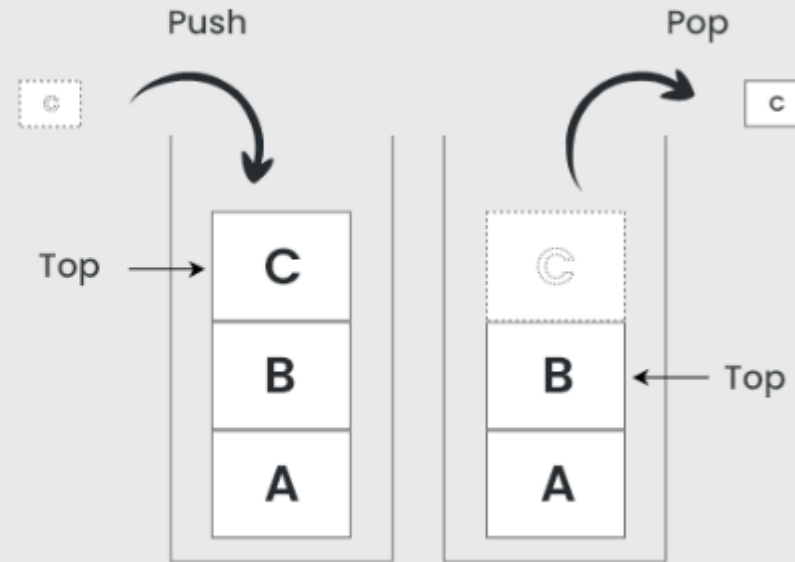
- In this form of data structure, the data elements don't stay arranged linearly or sequentially.
- A user can't traverse all of its elements in a single run.
- For Example
 1. Tree
 2. Graph

Stacks



Stack

Data Structure



Stacks

- What is a Stack?
- **Stack** is a linear data structure that follows **LIFO (Last In First Out) Principle**, the last element inserted is the first to be popped out.
- It means both insertion and deletion operations happen at one end only.
- In other words, a *stack can be defined as a container in which insertion and deletion can be done from the one end known as the top of the stack.*

Stacks

- **LIFO(Last In First Out) Principle**
- Here are some real world examples of LIFO
- Consider a stack of plates. When we add a plate, we add at the top. When we remove, we remove from the top.

Working of Stack

- Suppose we want to store the elements in a stack and let's assume that stack is empty.
- We have taken the stack of size 5 as shown below



Standard Stack Operations

• The following are some common operations implemented on the stack:

1. **push():**
2. **pop():**
3. **isEmpty():**
4. **isFull():**
5. **peek():**
6. **count():**
7. **change():**
8. **display():**

Standard Stack Operations

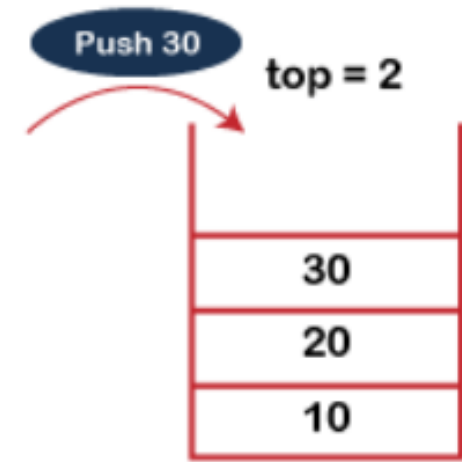
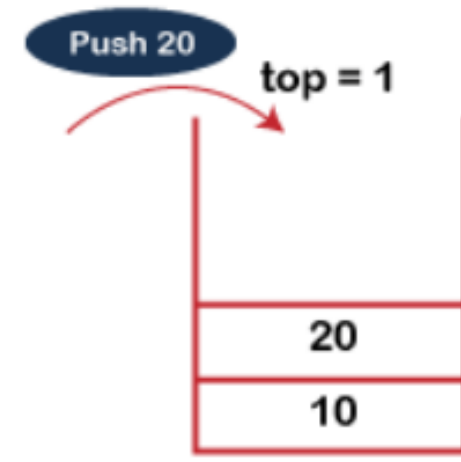
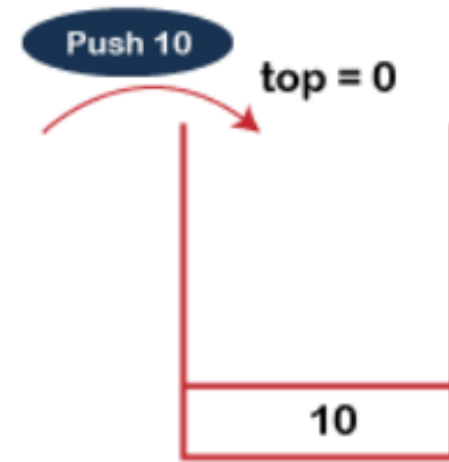
- **The following are some common operations implemented on the stack:**
- **push():** When we insert an element in a stack then the operation is known as a push. If the stack is full then the overflow condition occurs.
- **pop():** When we delete an element from the stack, the operation is known as a pop. If the stack is empty means that no element exists in the stack, this state is known as an underflow state.
- **isEmpty():** It determines whether the stack is empty or not.
- **isFull():** It determines whether the stack is full or not.'

Standard Stack Operations

- **peek()**: It returns the element at the top position without removing it.
- **count()**: It returns the total number of elements available in a stack.
- **display()**: It prints all the elements available in the stack.

PUSH operation

- **The steps involved in the PUSH operation is given below:**
- Before inserting an element in a stack, we check whether the stack is full.
- If the stack is full, then the ***overflow*** condition occurs.
- When the new element is pushed in a stack, first, the value of the top gets incremented, i.e., **$\text{top}=\text{top}+1$** , and the element will be placed at the new position of the **top**.

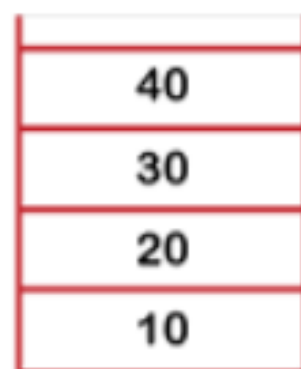


Stack is full

POP operation

- **The steps involved in the POP operation is given below:**
- Before deleting the element from the stack, we check whether the **stack is empty**.
- If we try to delete the element from the empty stack, then the ***underflow*** condition occurs.
- If the stack is not empty, we first access the element which is pointed by the ***top***. ***We process the element at top of stack.***
- Once the pop operation is performed, the top is decremented by 1, i.e., **$\text{top}=\text{top}-1$** .

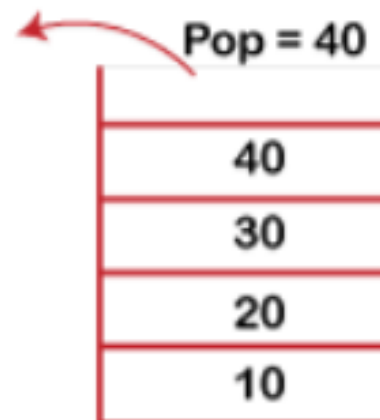
top = 3



Stack is full

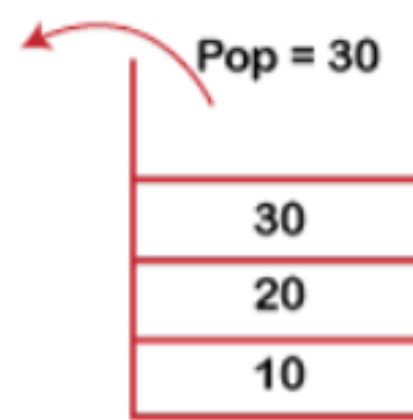
top = 2

Pop = 40



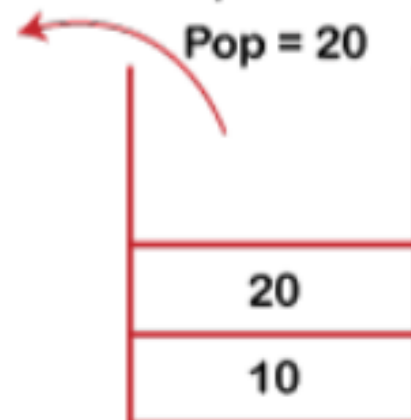
top = 1

Pop = 30



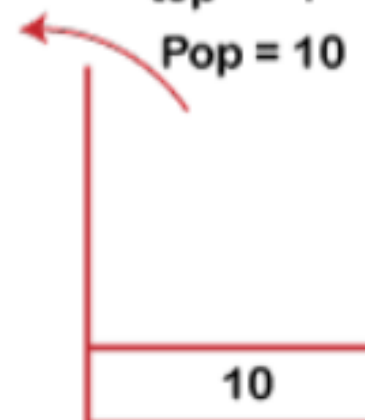
top = 0

Pop = 20



top = - 1

Pop = 10



top = - 1

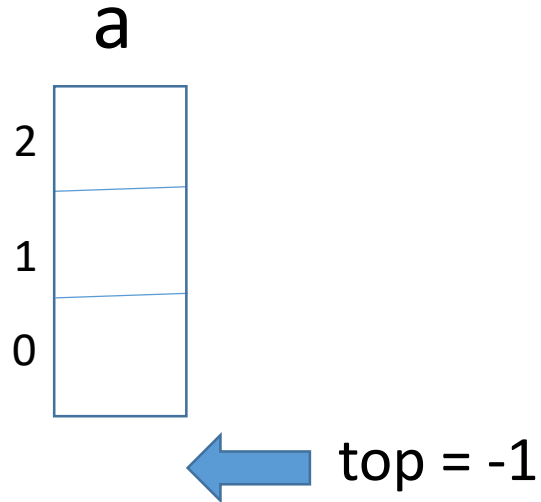
empty

Example of push and pop operation

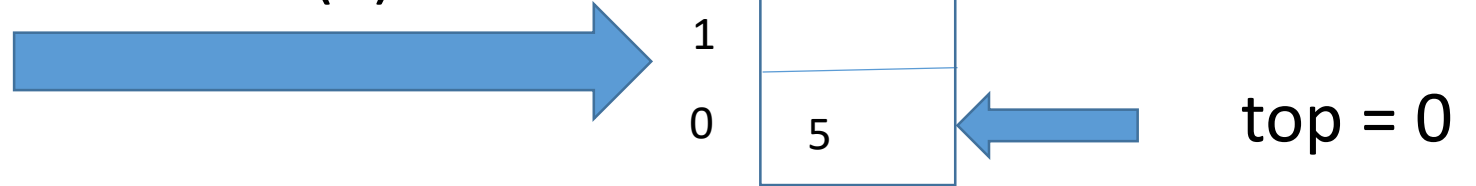
- Suppose there is an array `a` of size 3 i.e. `int a[3];`

Example of Push operation

Step-1



Push(5)

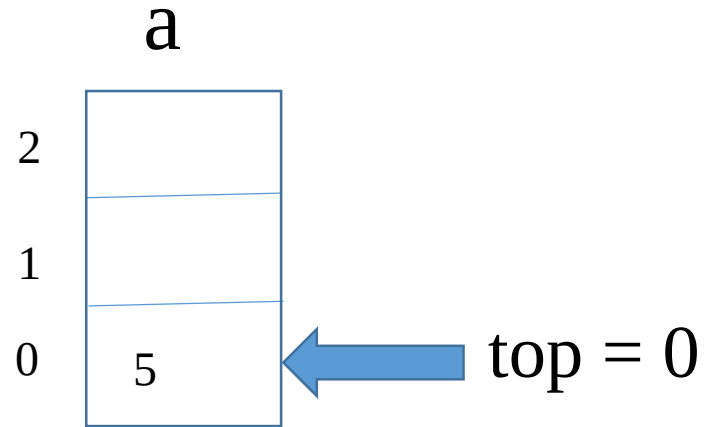


Initially stack is empty so
 $\text{top} = -1$

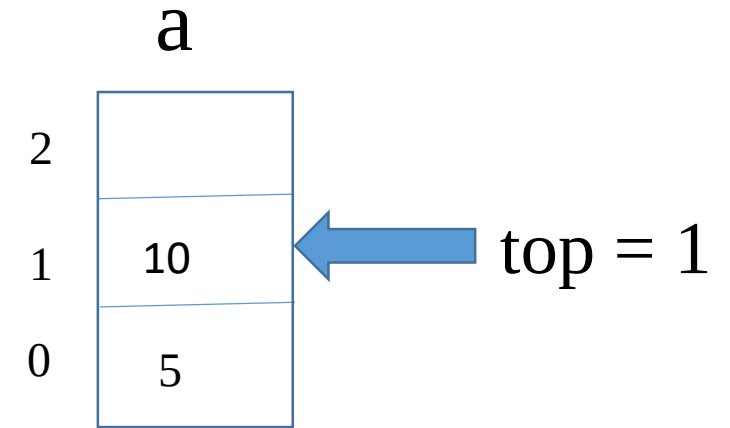
$\text{top} = \text{top} + 1 = -1 + 1 = 0$
 $\text{top} = 0$
 $a[\text{top}] = a[0] = 5$

Cont...

Step-2



Push(10)



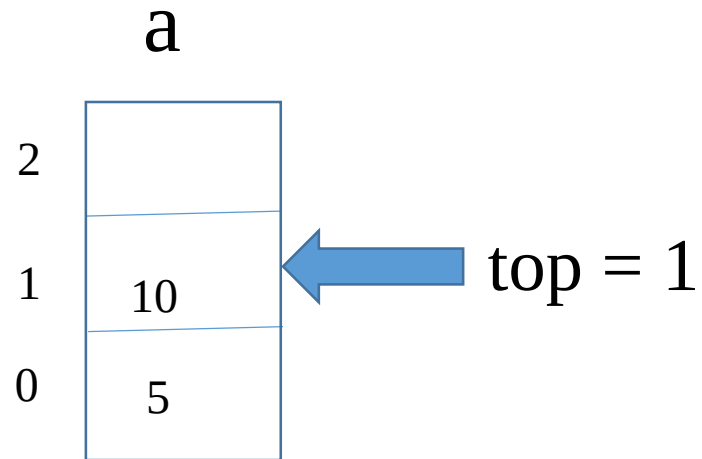
$\text{top} = \text{top} + 1 = 0 + 1 = 1$

$\text{top} = 1$

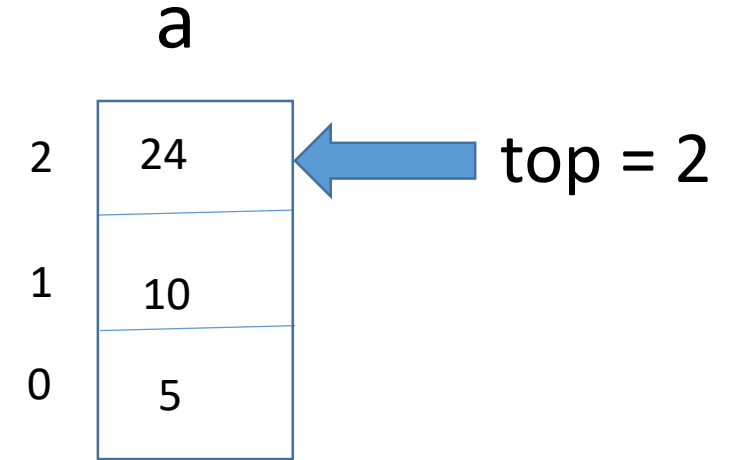
$a[\text{top}] = a[1] = 10$

Cont...

Step-3



Push(24)



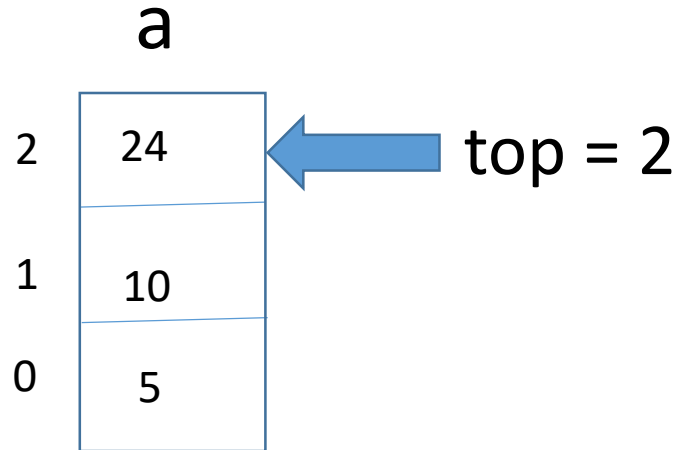
$\text{top} = \text{top} + 1 = 1 + 1 = 2$

$\text{top} = 2$

$a[\text{top}] = a[2] = 24$

Cont...

Step-4



Push(30)

As, stack is full, so we can not push (insert) element 30 and this condition is called stack overflow condition.

Note: Stack overflow condition is $\text{top} == \text{maxsize} - 1$

Where

maxsize = maximum size of an array.

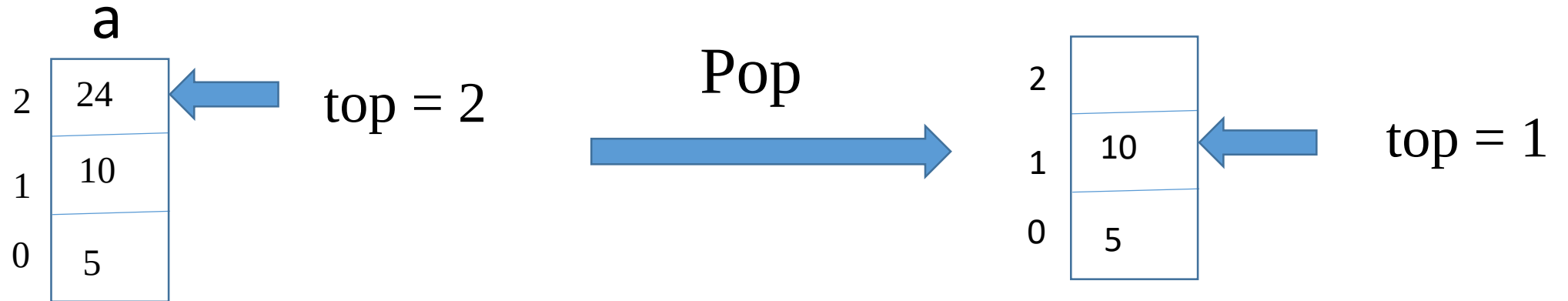
Here: $\text{top} = 2$, $\text{maxsize} = 3$.

$\text{maxsize} - 1 = 3 - 1 = 2$

$2 == 2$, it means stack is full and we can not push (insert) element 30 and this condition is called stack overflow condition.

Example of Pop operation

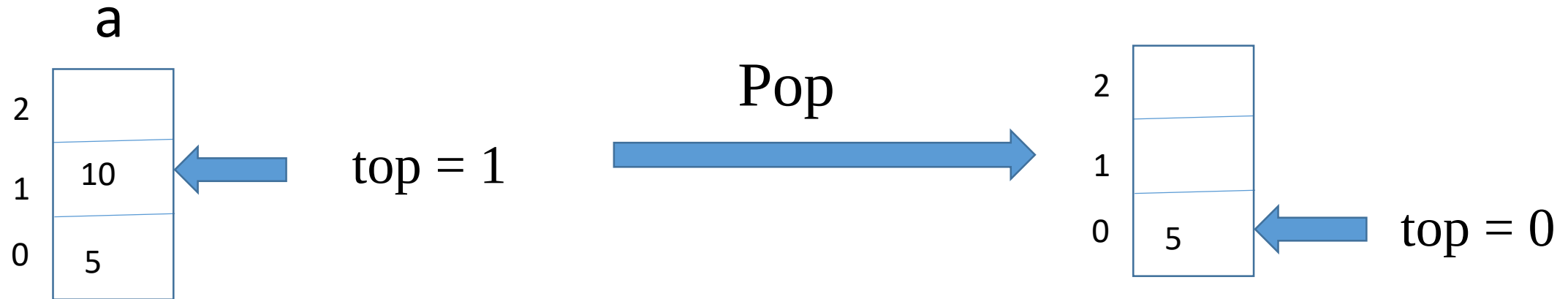
Step-1



$$\text{top} = \text{top} - 1 = 2 - 1 = 1$$
$$\text{top} = 1$$

Cont...

Step-2

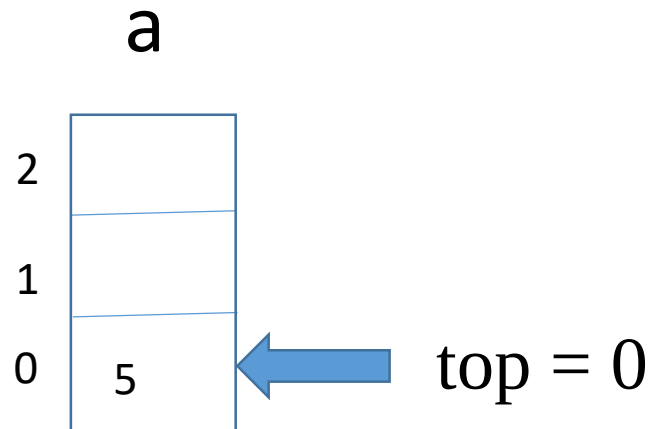


$$\text{top} = \text{top} - 1 = 1 - 1 = 0$$

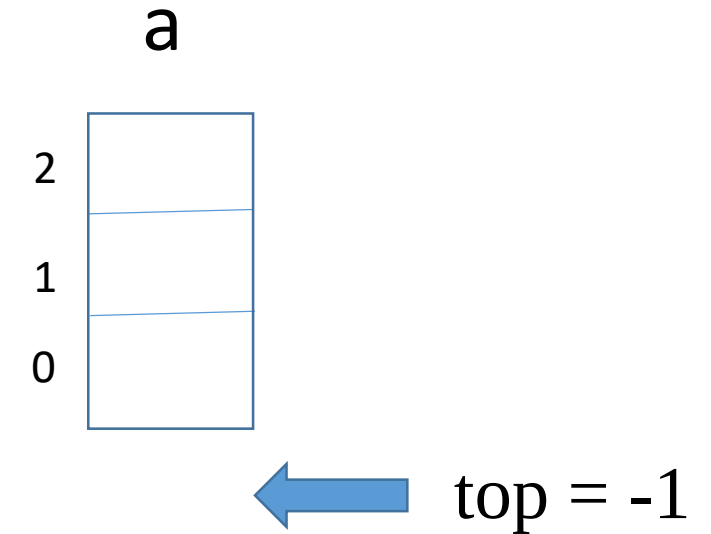
$$\text{top} = 0$$

Cont...

Step-3



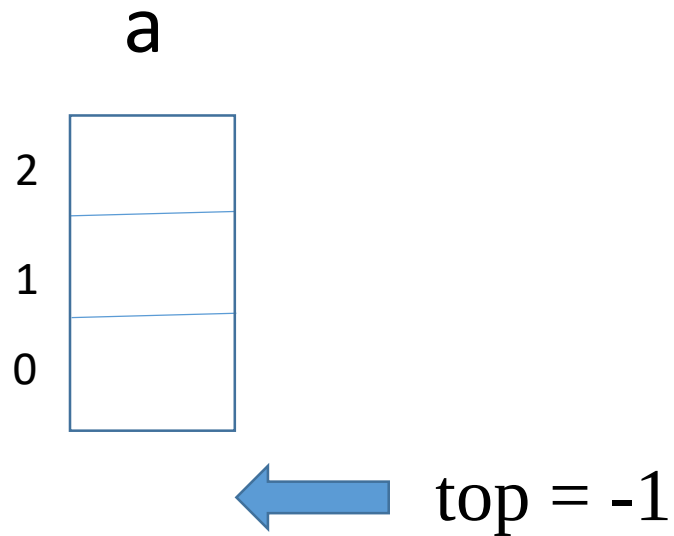
Pop



$\text{top} = \text{top} - 1 = 0 - 1 = -1$
 $\text{top} = -1$

Cont...

Step-4



Pop



Now, we can not pop (delete) a element from a top of a stack because stack is empty and this condition is called **stack under flow condition**.

Note: Stack under flow condition is $top == -1$

Stacks

- (1) Algorithm for push operation on stack
- (2) Algorithm for pop operation on stack.

Stacks : Algorithm for push operation

Procedure 6.1:

PUSH(STACK, TOP, MAXSTK, ITEM)

This procedure pushes an ITEM onto a stack.

1. [Stack already filled?]

If $TOP = MAXSTK$, then: Print: OVERFLOW, and Return.

2. Set $TOP := TOP + 1$. [Increases TOP by 1.]

3. Set $STACK[TOP] := ITEM$. [Inserts ITEM in new TOP position.]

4. Return.

Stacks : Algorithm for pop operation

Procedure 6.2:

: POP(STACK, TOP, ITEM)

This procedure deletes the top element of STACK and assigns it to the variable ITEM.

1. [Stack has an item to be removed?]
If $TOP = 0$, then: Print: UNDERFLOW, and Return.
2. Set $ITEM := STACK[TOP]$. [Assigns TOP element to ITEM.]
3. Set $TOP := TOP - 1$. [Decreases TOP by 1.]
4. Return.

Applications of Stack

- **Stack data structure has many applications. Some of them are as follows:**
 1. String reversal
 2. UNDO/REDO
 3. Recursion
 4. DFS(Depth First Search):
 5. Backtracking
 6. Expression conversion
 7. Memory management

Applications of Stack

- **String reversal:**

- Stack is also used for reversing a string. For example, we want to reverse a "ABCD" string, so we can achieve this with the help of a stack.
- First, we push all the characters of the string in a stack until we reach the null character.
- After pushing all the characters, we start taking out the character one by one until we reach the bottom of the stack.

- **UNDO/REDO:**

- It can also be used for performing UNDO/REDO operations. For example, we have an editor in which we write 'a', then 'b', and then 'c'; therefore, the text written in an editor is abc.

Applications of Stack

- **Recursion:**

- The recursion means that the function is calling itself again. To maintain the previous states, the compiler creates a system stack in which all the previous records of the function are maintained.

- **DFS(Depth First Search):**

- This search is implemented on a Graph, and Graph uses the stack data structure.

- **Backtracking:**

- Suppose we have to create a path to solve a maze problem. If we are moving in a particular path, and we realize that we come on the wrong way. In order to come at the beginning of the path to create a new path, we have to use the stack data structure.

Applications of Stack

- **Expression conversion:**

- Stack can also be used for expression conversion. This is one of the most important applications of stack. The list of the expression conversion is given below:
 - *Infix to postfix*
 - *Prefix to infix*

- **Memory management:**

- The stack manages the memory. The memory is assigned in the contiguous memory blocks. The memory is known as stack memory as all the variables are assigned in a function call stack memory. When the function is created, all its variables are assigned in the stack memory. When the function completed its execution, all the variables assigned in the stack are released

Applications of Stack

1. Convert Infix to Postfix notation
2. Evaluation of postfix notations using stack.

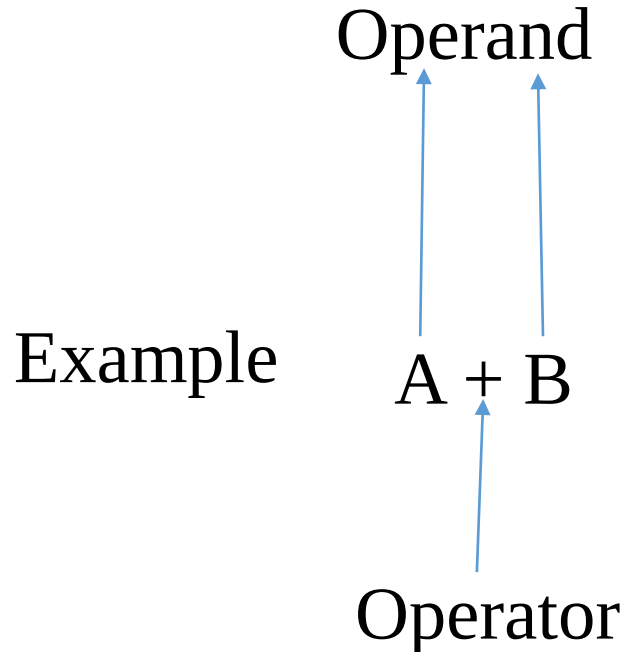
Notations of Arithmetic Expressions

Notations

- There are 3 types of notation.
 - Infix notation.
 - Prefix notation.
 - Postfix notation.
- An arithmetic expression can be represented in various form such as Infix form , prefix form and postfix form .

1. Infix notation (Infix form)

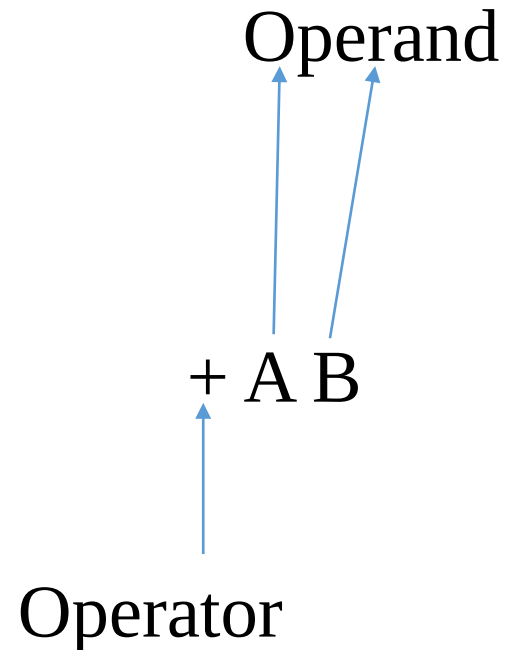
- In infix notation, operator is placed in middle of its operands.



2. Prefix notation (Prefix form)

- In prefix notation, operator is placed before its two operands. It is also called polish notation.

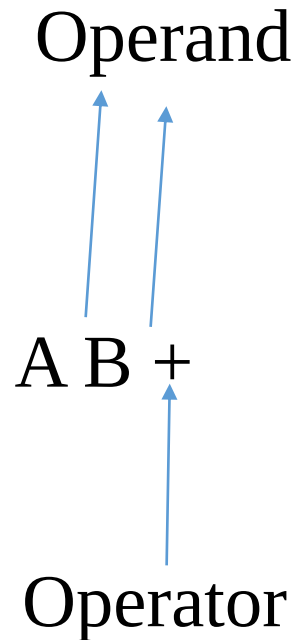
Example



3. Postfix notation (Postfix form)

- In postfix notation, operator is placed after its two operands. It is also called reverse polish notation or suffix notation.

- Example :



Stacks : What is infix notation?

- When the operator is written in between the operands, then it is known as **infix notation**. Operand does not have to be always a constant or a variable; it can also be an expression itself.
- **Syntax of infix notation is given below:**
- **<operand> <operator> <operand>**
- **For example**
- **A - B**
- **(P * 5)**
- **(p + q) * (r + s)**
- **4 + 6 * 2**

Postfix Expression

- The postfix expression is an expression in which the operator is written after the operands.
- For example, the postfix expression of infix notation (2+3) can be written as 23+.

Postfix expression

- Some key points regarding the postfix expression are:
- In postfix expression, operations are performed in the order in which they have written from left to right.
- It does not any require any parenthesis.
- We do not need to apply operator precedence rules and associativity rules.

Precedence

Precedence

Bracket () (Higher Precedence)

Exponential operator (^)

Multiplication / Division (* , /)

Addition / Subtraction (+, -) (Lower Precedence)



Application of Stack

1. Checking the validity of an Arithmetic Expression.
2. Evaluate (Solve) a postfix expression.
3. Converting infix expression into postfix expression.
4. Converting infix expression into prefix expression.

Evaluation of postfix notations using stack.

Evaluation of a Postfix Expression

Suppose P is an arithmetic expression written in postfix notation. The following algorithm, which uses a STACK to hold operands, evaluates P.

Algorithm 6.5: This algorithm finds the VALUE of an arithmetic expression P written in postfix notation.

1. Add a right parenthesis “)” at the end of P. [This acts as a sentinel.]
2. Scan P from left to right and repeat Steps 3 and 4 for each element of P until the sentinel “)” is encountered.
3. If an operand is encountered, put it on STACK.
4. If an operator $\otimes$ is encountered, then:
 - (a) Remove the two top elements of STACK, where A is the top element and B is the next-to-top element.
 - (b) Evaluate $B \otimes A$.
 - (c) Place the result of (b) back on STACK.[End of If structure.]
- [End of Step 2 loop.]
5. Set VALUE equal to the top element on STACK.
6. Exit.

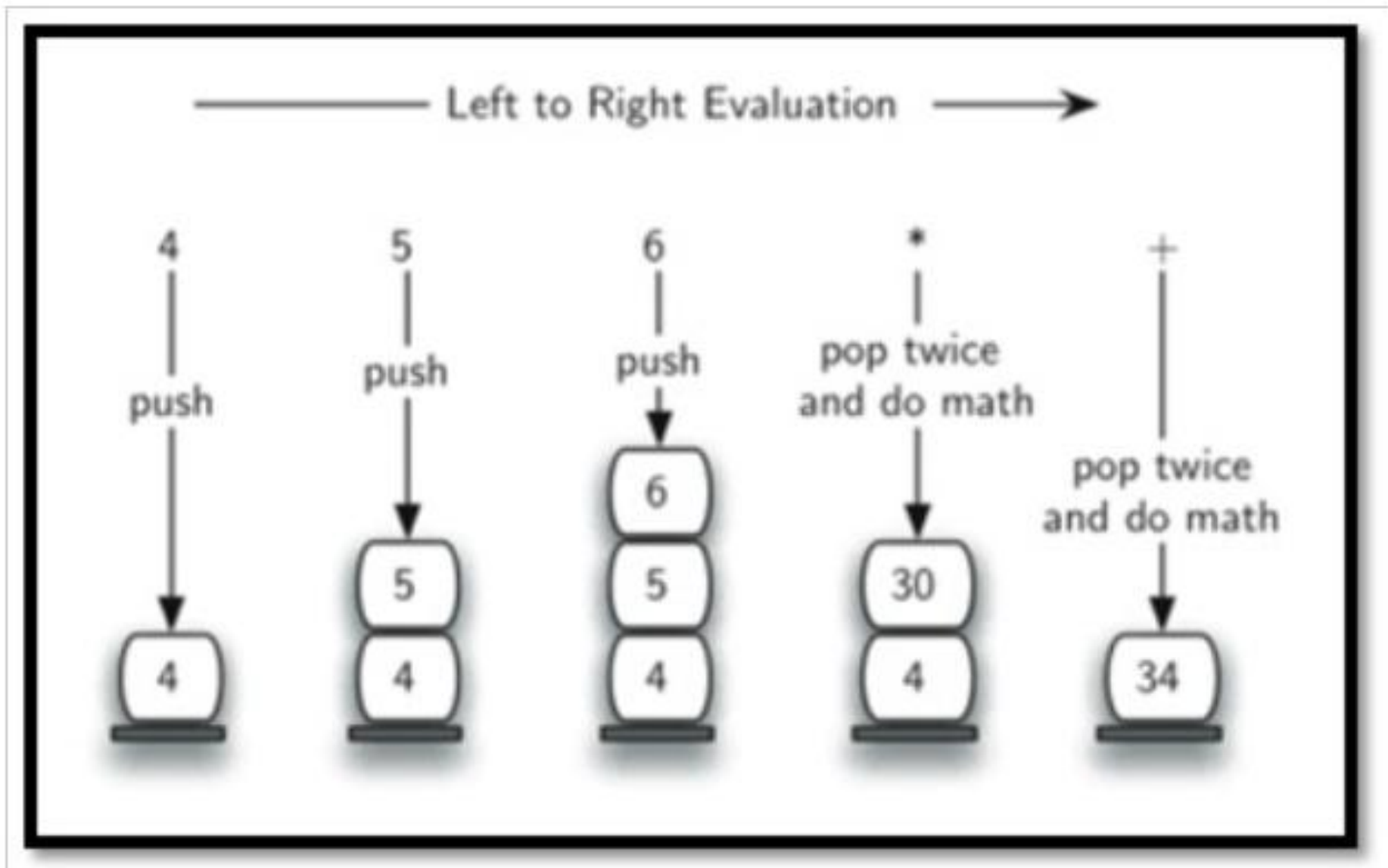
We note that, when Step 5 is executed, there should be only one number on STACK.

Algorithm :

Evaluation of postfix expression using stack.

1. Scan the expression from left to right.
2. If we encounter any operand in the expression, then we push the operand in the stack.
3. When we encounter any operator in the expression, then we pop the corresponding operands from the stack, Perform operation and push the result back on to stack.
4. When we finish with the scanning of the expression, the final value remains in the stack.

Expression: 456*+



Expression: 456*+

Step	Input Symbol	Operation	Stack	Calculation
1.	4	Push	4	
2.	5	Push	4,5	
3.	6	Push	4,5,6	
4.	*	Pop(2 elements) & Evaluate	4	$5*6=30$
5.		Push result(30)	4,30	
6.	+	Pop(2 elements) & Evaluate	Empty	$4+30=34$
7.		Push result(34)	34	
8.		No-more elements(pop)	Empty	34(Result)

Evaluate

Postfix Expression

4 5 7 2 + - ×

3 4 + 2 × 7 /

5 7 + 6 2 - ×

4 2 + 3 5 1 - × +

Postfix Expression

Result

 $4\ 5\ 7\ 2\ +\ -\ \times$ -16 $3\ 4\ +\ 2\ \times\ 7\ /$ 2 $5\ 7\ +\ 6\ 2\ -\ \times$ 48 $4\ 2\ +\ 3\ 5\ 1\ -\ \times\ +$ 18

Infix to Postfix expression

- **Converting infix expression into postfix expression by using stack**

Precedence

Precedence

Bracket () (Higher Precedence)

Exponential operator (^)

Multiplication / Division (* , /)

Addition / Subtraction (+, -) (Lower Precedence)



Algorithm 6.6: POLISH(Q, P)

Suppose Q is an arithmetic expression written in infix notation. This algorithm finds the equivalent postfix expression P.

1. Push "(" onto STACK, and add ")" to the end of Q.
 2. Scan Q from left to right and repeat Steps 3 to 6 for each element of Q until the STACK is empty:
 3. If an operand is encountered, add it to P.
 4. If a left parenthesis is encountered, push it onto STACK.
 5. If an operator $\otimes$ is encountered, then:
 - (a) Repeatedly pop from STACK and add to P each operator (on the top of STACK) which has the same precedence as or higher precedence than $\otimes$.
 - (b) Add $\otimes$ to STACK.[End of If structure.]
 6. If a right parenthesis is encountered, then:
 - (a) Repeatedly pop from STACK and add to P each operator (on the top of STACK) until a left parenthesis is encountered.
 - (b) Remove the left parenthesis. [Do not add the left parenthesis to P.][End of If structure.]
- [End of Step 2 loop.]
7. Exit.

Algorithm : Infix to Postfix expression

1. Start
 2. Push “(“ onto Stack, and add “)” to the end of X.
 3. Scan X from left to right and repeat Step 4 to 7 for each element of X until the Stack is empty.
 4. If an operand is encountered, add it to Y.
 5. If a left parenthesis is encountered, push it onto Stack.
 6. If an operator is encountered ,then:
 - i. Repeatedly pop from Stack and add to Y each operator (on the top of Stack) which has the same precedence as or higher precedence than operator.
 - ii. Add operator to Stack.
- [End Step-6]

Algorithm : Infix to Postfix expression...

7. If a right parenthesis is encountered ,then:
 - i. Repeatedly pop from Stack and add to Y each operator (on the top of Stack) until a left parenthesis is encountered.
 - ii. Remove the left Parenthesis.[End Step-7]
8. END.

Infix : $A + (B * C - (D / E ^ F) * G) * H$

P: A B C * D E F ↑ / G * - H * +
 which is the required postfix equivalent of Q.

Symbol Scanned	STACK	Expression P
(1) A	(	A
(2) +	(+	A
(3) (	(+ (	A
(4) B	(+ (	A B
(5) *	(+ (*	A B
(6) C	(+ (*	A B C
(7) -	(+ (-	A B C *
(8) (	(+ (- (	A B C *
(9) D	(+ (- (	A B C * D
(10) /	(+ (- (/	A B C * D
(11) E	(+ (- (/	A B C * D E
(12) ↑	(+ (- (/ ↑	A B C * D E E
(13) F	(+ (- (/ ↑	A B C * D E F
(14))	(+ (-	A B C * D E F ↑ /
(15) *	(+ (- *	A B C * D E F ↑ /
(16) G	(+ (- *	A B C * D E F ↑ / G
(17))	(+ (- *	A B C * D E F ↑ / G * -
(18) *	(+ (- *	A B C * D E F ↑ / G * - H
(19) H	(+ (- *	A B C * D E F ↑ / G * - H * +
(20))		

Postfix : $A B C * D E F ^ / G * - H * +$

Infix : $A + (B * C - (D / E ^ F) * G) * H \quad)$

	Symbol	Stack	Postfix
		(	
1	A	(	A
2	+	(+	A
3	(	(+ (	A
4	B	(+ (	A B
5	*	(+ (*	A B
6	C	(+ (*	A B C
7	-	(+ (-	A B C *
8	(	(+ (- (	A B C *
9	D	(+ (- (	A B C * D
10	/	(+ (- (/	A B C * D E
11	E	(+ (- (/	A B C * D E
12	^	(+ (- (/ ^	A B C * D E
13	F	(+ (- (/ ^	A B C * D E F
14	)	(+ (-	A B C * D E F ^ /
15	*	(+ (- *	A B C * D E F ^ /
16	G	(+ (- *	A B C * D E F ^ / G
17	)	(+	A B C * D E F ^ / G * -
18	*	(+ *	A B C * D E F ^ / G * -
19	H	(+ *	A B C * D E F ^ / G * - H
20	)	Empty	A B C * D E F ^ / G * - H * +

Infix Expression: $(a*b+c*(d-e))$.

Sl.	Scanned	STACK	Postfix Expression	Description
1.	(	(		START
2.	a	(	a	
3.	*	(*	a	
4.	b	(*	ab	
5.	+	(+	ab*	'*' is at higher precedence than '+',
6.	c	(+	ab*c	
7.	*	(*	ab*c	
8.	(	(*	ab*c	
9.	d	(*	ab*cd	
10.	-	(*	ab*cd	
11.	e	(*	ab*cde	
12.	)	(*	ab*cde-	POP from top of the STACK that's why '*' come first.
13.	)	EMPTY	ab*cde-*+	END

Resultant Postfix Expression: $ab*cde-*+$

$$A - (B / C + (D \% E * F) / G) * H$$

$$A - (B / C + (D \% E * F) / G) * H$$

Infix Character Scanned	Stack	Postfix Expression
A		A
-	-	A
(	- (	A
B	- (	A B
/	- (/	A B
C	- (/	A B C
+	- (+	A B C /
(	- (+ (	A B C /
D	- (+ (	A B C / D
%	- (+ (%	A B C / D
E	- (+ (%	A B C / D E
*	- (+ (% *	A B C / D E
F	- (+ (% *	A B C / D E F
)	- (+	A B C / D E F * %
/	- (+ /	A B C / D E F * %
G	- (+ /	A B C / D E F * % G
)	-	A B C / D E F * % G / +
*	- *	A B C / D E F * % G / +
H	- *	A B C / D E F * % G / + H
		A B C / D E F * % G / + H * -

Question 1. The postfix form of the expression $(A + B) * (C * D - E) * F / G$ is?

- a) $AB + CD * E - FG / **$
- b) $AB + CD * E - F ** G /$
- c) $AB + CD * E - * F * G /$
- d) $AB + CDE * - * F * G /$

Question 1. The postfix form of the expression $(A + B) * (C * D - E) * F / G$ is?

a) $AB + CD * E - FG / **$

b) $AB + CD * E - F ** G /$

c) $AB + CD * E - * F * G /$

d) $AB + CDE * - * F * G /$

ANSWER: a) $AB + CD * E - FG / **$

Infix expression	Postfix expression
$(A + B) * (C - D)$	
$(A + B) / (C - D) - (E * F)$	

Infix expression	Postfix expression
$(A + B) * (C - D)$	$AB + CD - *$
$(A + B) / (C - D) - (E * F)$	$AB + CD - / EF * -$

Infix to Postfix expression

Q. Solve the $A + ((B + C) + (D + E) * F) / G$ infix expression into postfix expression by using stack.

Ans: $ABC + DE + F * + G / +$

Infix to Postfix expression

Q. Converting following infix expression $a+(b*c-(d/e)*g)*h$ into postfix expression by using stack.

Ans: **$abc*de/g*-h*+$**

Infix to Postfix expression

Q: Converting $a+(b*c)$ expression into postfix expression.

Ans: $abc*+$

References

- <https://www.javatpoint.com/data-structure-stack>